TÊN TRƯỜNG ĐẠI HỌC

KHOA / VIỆN CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

ĐẾM ĐƯỜNG ĐI THỜI GIAN TRÊN RỪNG, ĐỒ THỊ SERIES–PARALLEL VÀ ĐỒ THỊ CÓ TREEWIDTH BỊ CHẶN

Sinh viên thực hiện: **TÊN SINH VIÊN**
Mã số sinh viên: MÃ SỐ SINH VIÊN
Giảng viên hướng dẫn: **TÊN GIẢNG VIÊN HƯỚNG DẪN**
Chương trình: CỬ NHÂN / KỸ SƯ CÔNG NGHỆ THÔNG TIN

Thành phố Hồ Chí Minh, 2026

Lời cam đoan

Tôi cam đoan khóa luận này trình bày trung thực quá trình nghiên cứu, thiết kế thuật toán, cài đặt và thực nghiệm của tôi dưới sự hướng dẫn của giảng viên hướng dẫn. Các kết quả từ công trình của tác giả khác đều được ghi nguồn. Phần mềm, tập sinh dữ liệu và kịch bản thực nghiệm đi kèm khóa luận cho phép tái lập các kết quả được báo cáo.

Kết quả về đồ thị series–parallel và treewidth trong khóa luận là kết quả nghiên cứu đang chờ thẩm định độc lập; khóa luận không tuyên bố kết quả này đã qua bình duyệt. Các số liệu thực nghiệm được mô tả đúng phạm vi đã chạy và không được gọi là bộ benchmark chính thức của bài báo gốc.

TÊN SINH VIÊN

Ngày .....tháng .....năm 2026

Lời cảm ơn

Tôi xin gửi lời cảm ơn chân thành đến TÊN GIẢNG VIÊN HƯỚNG DẪN vì sự hướng dẫn, phản biện và hỗ trợ trong suốt quá trình thực hiện khóa luận. Tôi cảm ơn các thầy cô tại KHOA / VIỆN CÔNG NGHỆ THÔNG TIN đã cung cấp nền tảng về thuật toán, lý thuyết đồ thị và phương pháp nghiên cứu khoa học. Tôi cũng cảm ơn gia đình và bạn bè đã động viên trong quá trình xây dựng chứng minh, cài đặt phần mềm và thực hiện các chiến dịch kiểm thử.

Khóa luận kế thừa câu hỏi mở được Enright, Meeks và Molter nêu trong công trình *Counting Temporal Paths*. Tôi trân trọng giá trị của công trình này trong việc định hình bài toán, thuật ngữ và bối cảnh độ phức tạp cho nghiên cứu hiện tại.

Tóm tắt

Khóa luận nghiên cứu bài toán đếm số đường đi thời gian đơn từ đỉnh nguồn s đến đỉnh đích z trong đồ thị thời gian vô hướng, trong đó nhãn thời gian dọc theo đường đi không giảm. Bài toán tổng quát là khó; công trình của Enright, Meeks và Molter chứng minh thuật toán $\mathcal{O}(|V|T^2)$ khi đồ thị nền là rừng, đưa ra thuật toán tham số cố định theo tổ hợp treewidth và lifetime bằng logic MSO, đồng thời đặt giả thuyết rằng trường hợp series–parallel có thể giải trong thời gian đa thức.

Khóa luận có ba đóng góp thuật toán. Thứ nhất, truy hồi trên rừng được đánh giá lại bằng tổng tiền tố trong $\mathcal{O}(|P|r)$ phép toán số học, hoặc

$$\mathcal{O}\left(|P| + \sum_{e \in P} |L(e)|\right)$$

khi dùng danh sách nhãn thưa đã sắp xếp; ở đây P là đường đi tĩnh duy nhất và r là số hạng thời gian phân biệt sau nén hạng. Thứ hai, đối với đồ thị two-terminal series–parallel có hai cực trùng với truy vấn, khóa luận xây dựng ma trận truyền thời gian: phép ghép song song trở thành cộng ma trận và phép ghép nối tiếp trở thành nhân ma trận, cho thuật toán chính xác $\mathcal{O}(mr^3)$. Thứ ba, khóa luận trình bày một quy hoạch động kết nối tường minh trên phân rã cây, chạy trong

$$Nr^{\mathcal{O}(w)}2^{\mathcal{O}(w \log w)}$$

phép toán số học khi được cho phân rã có độ rộng w. Do đó, bài toán thuộc XP theo treewidth của đồ thị nền. Vì đồ thị series–parallel có treewidth không quá 2, kết quả suy ra thuật toán đa thức cho cặp đầu cuối tùy ý, qua đó giải giả thuyết series–parallel với điều kiện chứng minh và tính mới vượt qua thẩm định độc lập.

Các thuật toán được cài đặt bằng C++20 với số nguyên chính xác độ dài tùy ý. Bộ kiểm thử có phương pháp tham chiếu độc lập, sinh dữ liệu xác định từ seed và lưu chỉ số trạng thái. Chiến dịch hiện tại thực hiện $6\,500$ đối chiếu ngẫu nhiên trên rừng, TTSP và partial 2-tree mà không ghi nhận sai lệch. Toàn bộ 45 cấu hình diamond-chain bắt buộc đến 256 diamond và 32 hạng thời gian đều hoàn thành. Kết quả cho thấy xu hướng tăng tốc phù hợp với phân tích lý thuyết, nhưng khóa luận ghi nhận rõ rằng chiến dịch Graph Atlas và kiểm thử vét cạn nhãn đến $n \leq 5$ vẫn là điều kiện phát

hành còn lại.

Từ khóa: đồ thị thời gian; đếm đường đi; series–parallel; treewidth; quy hoạch động; thuật toán tham số hóa.

Abstract

This thesis studies the exact counting of simple non-strict temporal s–z paths in undirected temporal graphs. It improves the forest recurrence from $\mathcal{O}(|V|T^2)$ to $\mathcal{O}(|P|r)$, and to input-linear time on sorted sparse labels. It develops an exact transfer-matrix algorithm for two-terminal series–parallel graphs and an explicit connectivity dynamic program running in $Nr^{\mathcal{O}(w)}2^{\mathcal{O}(w \log w)}$ arithmetic operations on a supplied width-w tree decomposition. The latter places the problem in XP parameterized by underlying treewidth and implies polynomial-time solvability on series–parallel graphs. A C++20 implementation with arbitrary-precision arithmetic is validated against independent enumeration on $6\,500$ random instances with zero mismatches. The series–parallel theorem is presented as a research result pending independent proof and novelty review.

Mục lục

Lời cam đoan **i**

Lời cảm ơn **ii**

Tóm tắt **iii**

Abstract **v**

Danh mục ký hiệu và chữ viết tắt **xi**

1 Giới thiệu **1**
1.1 Bối cảnh và động lực 1
1.2 Phát biểu vấn đề 2
1.3 Đóng góp 2
1.4 Phạm vi và mức độ khẳng định 3
1.5 Cấu trúc khóa luận 4

2 Cơ sở lý thuyết và công trình liên quan **5**
2.1 Đồ thị thời gian 5
2.2 Đếm chính xác và độ phức tạp tham số 5
2.3 Nén hạng thời gian 6
2.4 Rừng, series–parallel và treewidth 6
2.5 Kết quả đã biết và khoảng trống nghiên cứu 7
2.6 Vị trí của đóng góp mới 8

3 Thuật toán nhanh trên rừng **9**
3.1 Truy hồi gốc 9
3.2 Quy hoạch động tổng tiền tố 9
3.3 Quy hoạch động thưa 10
3.4 Oracle khép kín 11
3.5 Ý nghĩa thuật toán 11

4 Đại số truyền cho đồ thị TTSP **12**
4.1 Tại sao không thể liệt kê đường đi tĩnh 12
4.2 Ma trận truyền thời gian 12
4.3 Ghép song song 12
4.4 Ghép nối tiếp 13

4.5 Ví dụ diamond 13
4.6 Giới hạn của thuật toán chuyên biệt 14
4.7 Hướng tối ưu hóa 14

5 Quy hoạch động theo treewidth 15
5.1 Ý tưởng cục bộ hóa điều kiện thời gian 15
5.2 Chuẩn hóa phân rã 15
5.3 Bất biến nghiệm bộ phận 16
5.4 Không gian trạng thái 16
5.5 Các chuyển trạng thái đơn 16
5.5.1 Leaf và introduce-vertex 16
5.5.2 Introduce-edge 17
5.5.3 Forget-vertex 17
5.6 Nút join và bổ đề incidence 17
5.7 Điều kiện chấp nhận 18
5.8 Chứng minh đúng 18
5.9 Độ phức tạp 19

6 Thiết kế và cài đặt hệ thống 20
6.1 Mục tiêu kỹ thuật 20
6.2 Kiến trúc repository 20
6.3 Biểu diễn đồ thị và nén hạng 20
6.4 Số nguyên chính xác 21
6.5 Mã hóa trạng thái treewidth 21
6.6 CLI và telemetry 21
6.7 Phương pháp tham chiếu độc lập 22
6.8 Sinh dữ liệu và seed 22
6.9 Quy trình thất bại 22

7 Đánh giá thực nghiệm 24
7.1 Câu hỏi và giả thuyết thực nghiệm 24
7.2 Môi trường 24
7.3 Kiểm chứng tính đúng 24
7.4 So sánh forest 25
7.5 Diamond-chain TTSP 26
7.6 Treewidth DP 27
7.7 Đe dọa tới tính hợp lệ 27
7.8 Kết luận thực nghiệm 28

8 Kết luận và hướng phát triển **29**
8.1 Tổng kết kết quả 29
8.2 Trả lời câu hỏi nghiên cứu 29
8.3 Đóng góp đối với giả thuyết series–parallel 30
8.4 Hạn chế 30
8.5 Hướng phát triển 31
8.6 Kết lời 31

A Định dạng dữ liệu và decomposition **32**
A.1 Temporal graph 32
A.2 TTSP expression 32
A.3 Nice tree decomposition 33
A.4 Schema kết quả 33

B Hướng dẫn tái lập **34**
B.1 Build 34
B.2 Chiến dịch 34
B.3 Biên dịch khóa luận 34

C Checklist thẩm định chứng minh **36**

Danh mục hình

4.1 Một diamond là phép ghép song song của hai nhánh nối tiếp. 14

7.1 Xu hướng thời gian forest; nhãn trục x là $|P|/r$. Trục y logarithmic. 26
7.2 Thời gian TTSP theo k tại $r = 32$, lấy trực tiếp từ bảng CSV. . . 26

Danh mục bảng

2.1 Vị trí của các kết quả trong khóa luận. 8

6.1 Các thành phần chính của phần mềm. 20

7.1 Kết quả đối chiếu chính xác với phương pháp tham chiếu. . . 25
7.2 Forest dense smoke benchmark. 25

Danh mục ký hiệu và chữ viết tắt

V, E	Tập đỉnh và tập cạnh của đồ thị nền tĩnh.
$\mathcal{E}$	Tập cạnh–thời gian (e, t).
T	Lifetime, tức nhãn thời gian lớn nhất trong biểu diễn gốc.
r	Số giá trị thời gian phân biệt sau nén hạng.
$L(e)$	Danh sách hạng thời gian hoạt động của cạnh tĩnh e.
s, z	Đỉnh nguồn và đỉnh đích của truy vấn.
N	Đại lượng kích thước đầu vào $\lvert V\rvert + \lvert\mathcal{E}\rvert$.
w	Treewidth của đồ thị nền.
TTSP	Two-terminal series–parallel.
TD	Tree decomposition, phân rã cây.
MSO	Monadic second-order logic.
FPT	Fixed-parameter tractable.
XP	Lớp bài toán có thời gian $N^{f(k)}$ theo tham số k.
DP	Dynamic programming, quy hoạch động.
RSS	Resident set size, bộ nhớ vật lý cực đại được báo cáo.

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực

Nhiều mạng thực tế không bất biến theo thời gian. Liên kết giữa hai người chỉ tồn tại khi họ gặp nhau; một chuyến xe chỉ nối hai địa điểm tại một thời điểm cụ thể; một liên kết truyền thông chỉ khả dụng trong một khoảng thời gian. Mô hình đồ thị tĩnh làm mất thứ tự của các tương tác và có thể kết luận sai rằng một hành trình tồn tại dù các cạnh của hành trình xuất hiện theo thứ tự không khả thi. Đồ thị thời gian khắc phục vấn đề này bằng cách gắn thời điểm hoạt động cho mỗi cạnh.

Một đường đi thời gian không chỉ phải là đường đi trong đồ thị nền mà còn phải tôn trọng thứ tự thời gian. Việc tìm một đường đi như vậy thường dễ hơn đáng kể so với việc đếm tất cả đường đi. Số lượng đường đi liên quan trực tiếp tới độ dư thừa của kết nối, phân tích độ tin cậy và các đại lượng trung tâm dựa trên đường đi. Công trình *Counting Temporal Paths* của Enright, Meeks và Molter khởi đầu nghiên cứu có hệ thống về độ phức tạp tham số và xấp xỉ của bài toán này [1].

Bài báo chứng minh bài toán khó ngay cả dưới những hạn chế cấu trúc mạnh, nhưng cũng chỉ ra một số miền khả giải. Đặc biệt, khi đồ thị nền là rừng, đường đi tĩnh giữa hai đỉnh là duy nhất và một truy hồi theo thời gian cho thuật toán $\mathcal{O}(|V|T^2)$. Khi tham số hóa đồng thời bởi treewidth và lifetime, bài toán là FPT nhờ một biểu diễn MSO và định lý Courcelle. Trong phần kết luận, các tác giả dự đoán rằng thuật toán đa thức trên rừng có thể mở rộng sang đồ thị series–parallel [1].

Câu hỏi này hấp dẫn vì series–parallel là lớp đầu tiên vượt ra ngoài rừng nhưng vẫn có cấu trúc phân rã chặt. Ghép song song tạo ra nhiều tuyến tĩnh, thậm chí số tuyến tăng theo hàm mũ trong kích thước biểu diễn. Vì vậy, liệt kê đường đi không còn là một chiến lược đa thức. Đồng thời, treewidth của lớp này không quá 2, gợi ý rằng một trạng thái cục bộ đủ giàu có thể nén được toàn bộ lịch sử đường đi.

1.2 Phát biểu vấn đề

Cho đồ thị thời gian vô hướng $\mathcal{G} = (V, \mathcal{E}, T)$ và hai đỉnh phân biệt $s, z \in V$. Mỗi phần tử của $\mathcal{E}$ có dạng $(\{u, v\}, t)$ với $t \in [T]$. Một đường đi thời gian không nghiêm ngặt là một đường đi tĩnh đơn

$$P = (v_0 = s, v_1, \ldots, v_\ell = z)$$

cùng lựa chọn thời điểm $t_i \in L(\{v_{i-1}, v_i\})$ sao cho

$$t_1 \leq t_2 \leq \cdots \leq t_\ell.$$

Bài toán **#Temporal Path** yêu cầu đếm chính xác tất cả các đối tượng như vậy. Hai lựa chọn nhãn thời gian khác nhau trên cùng đường đi tĩnh được tính là hai đường đi thời gian khác nhau.

Khóa luận tập trung vào bốn câu hỏi nghiên cứu:

RQ1. Có thể đánh giá truy hồi trên rừng nhanh hơn cận $\mathcal{O}(|V|T^2)$ đã công bố hay không?

RQ2. Có thể xây dựng phép đại số truyền chính xác cho phân rã TTSP mà không liệt kê số đường đi tĩnh tăng theo hàm mũ hay không?

RQ3. Có thể đưa ra thuật toán tường minh tham số hóa chỉ bởi treewidth của đồ thị nền, thay cho meta-algorithm phụ thuộc đồng thời vào lifetime hay không?

RQ4. Cài đặt thực tế có đồng ý với các oracle khép kín và phương pháp brute force độc lập hay không, và xu hướng thời gian chạy có phù hợp với phân tích hay không?

1.3 Đóng góp

Khóa luận đưa ra các đóng góp sau.

C1. Chứng minh nén hạng thời gian bảo toàn song ánh tập đường đi thời gian. Trên rừng, sử dụng tổng tiền tố để giảm thời gian xuống $\mathcal{O}(|P|r)$ và sử dụng hai con trỏ trên danh sách thưa để đạt

$$\mathcal{O}\left(|P| + \sum_{e \in P} |L(e)|\right).$$

C2. Xây dựng ma trận truyền thời gian cho mỗi thành phần TTSP. Ghép song song là cộng ma trận, ghép nối tiếp là nhân ma trận. Thuật toán chạy trong $\mathcal{O}(mr^3)$ phép toán số học và áp dụng khi hai cực của phân rã chính là (s, z).

C3. Xây dựng quy hoạch động kết nối trên nice tree decomposition. Trạng thái ghi bậc có hướng kèm thời điểm cục bộ và phân hoạch liên thông. Một kiểm tra đồ thị incidence hai phía tại nút join ngăn chu trình, kể cả chu trình độ dài hai do cạnh song song trong incidence multigraph.

C4. Chứng minh thuật toán treewidth chạy trong

$$Nr^{\mathcal{O}(w)}2^{\mathcal{O}(w \log w)}$$

phép toán số học. Vì vậy, **#Temporal Path** thuộc XP theo treewidth. Hệ quả là bài toán có thời gian đa thức trên mọi lớp treewidth cố định, bao gồm series–parallel.

C5. Cung cấp cài đặt C++20 có số nguyên độ dài tùy ý, parser nén hạng, validator độc lập, bộ sinh xác định và benchmark có thể tái lập. Chiến dịch hiện tại có $6\,500$ đối chiếu không sai lệch và hoàn thành grid diamond-chain bắt buộc.

1.4 Phạm vi và mức độ khẳng định

Khóa luận chỉ xét đồ thị thời gian vô hướng và điều kiện thời gian không nghiêm ngặt. Biến thể nghiêm ngặt dùng dấu $<$ có vẻ chỉ đòi hỏi thay đổi so sánh cục bộ, nhưng không được tuyên bố là đã kiểm chứng. Thuật toán tổng quát nhận một phân rã cây hợp lệ làm đầu vào; việc tìm phân rã tối ưu là bài toán tiền xử lý tách biệt.

Kết quả series–parallel là đóng góp nghiên cứu của khóa luận, không phải kết quả từ bài báo 2025. Bằng chứng thực nghiệm không thay thế chứng minh, và chứng minh nội bộ không thay thế bình duyệt. Do đó, cách diễn đạt an toàn là: khóa luận *đề xuất và cài đặt* một lời giải đa thức cho trường hợp series–parallel, giải giả thuyết đã nêu *với điều kiện* kết quả vượt qua thẩm định độc lập và kiểm tra tính mới.

1.5 Cấu trúc khóa luận

Chương 2 trình bày nền tảng và công trình liên quan. Chương 3 phát triển hai cải tiến trên rừng. Chương 4 đưa ra đại số truyền TTSP. Chương 5 trình bày thuật toán treewidth, chứng minh đúng và hệ quả series–parallel. Chương 6 mô tả cài đặt và khả năng tái lập. Chương 7 trình bày thực nghiệm và đe dọa tới tính hợp lệ. Cuối cùng, Chương 8 tổng kết câu trả lời cho các câu hỏi nghiên cứu và hướng phát triển.

Chương 2

Cơ sở lý thuyết và công trình liên quan

2.1 Đồ thị thời gian

Định nghĩa 2.1 (Đồ thị thời gian)**.** Một đồ thị thời gian vô hướng, đơn là bộ ba $\mathcal{G} = (V, \mathcal{E}, T)$, trong đó V là tập đỉnh, $T \in \mathbb{N}$ là lifetime và

$$\mathcal{E} \subseteq \binom{V}{2} \times [T]$$

là tập cạnh–thời gian. Đồ thị nền $G = (V, E)$ chứa cạnh e khi tồn tại ít nhất một t sao cho $(e, t) \in \mathcal{E}$.

Với cạnh tĩnh e, ký hiệu $L(e) = \{t : (e, t) \in \mathcal{E}\}$. Một đường đi thời gian phải đơn về đỉnh ở lớp tĩnh; do đó, nó khác với temporal walk, nơi một đỉnh có thể xuất hiện nhiều lần. Khóa luận dùng quy ước không nghiêm ngặt của Enright và cộng sự: các nhãn liên tiếp chỉ cần không giảm [1]. Việc cho phép hai cạnh liên tiếp có cùng thời điểm là quan trọng trong cả chứng minh và kiểm thử.

Ví dụ 2.2. Xét đường đi tĩnh $s - u - z$ với $L(su) = \{1, 2\}$ và $L(uz) = \{2, 3\}$. Bốn lựa chọn hợp lệ là $(1, 2), (1, 3), (2, 2), (2, 3)$. Nếu dùng định nghĩa nghiêm ngặt, lựa chọn $(2, 2)$ bị loại và đáp án còn ba.

2.2 Đếm chính xác và độ phức tạp tham số

Giá trị đếm có thể lớn theo hàm mũ, nên thời gian chạy cần được hiểu theo cả số phép toán số học và độ dài bit. Với mỗi cạnh tĩnh, một nghiệm cục bộ có tối đa $1 + 2r$ lựa chọn gồm bỏ qua hoặc chọn một trong hai hướng và một hạng thời gian. Một cận thô cho giá trị trung gian là $(1 + 2r)^{|E|}$, tương ứng $\mathcal{O}(|E| \log r)$ bit. Vì vậy, với treewidth cố định, chi phí bit của số nguyên lớn vẫn là đa thức.

Một bài toán tham số hóa bởi k thuộc FPT nếu giải được trong $f(k)N^{\mathcal{O}(1)}$; nó thuộc XP nếu thời gian là $N^{f(k)}$. Khác biệt này có ý nghĩa ở đây. Bài

báo gốc cho FPT theo *tổ hợp* treewidth và lifetime bằng MSO/Courcelle [1, 3]. Thuật toán của khóa luận bỏ lifetime khỏi tập tham số nhưng đạt XP, không phải FPT, theo treewidth. Điều này không mâu thuẫn với kết quả hardness theo feedback vertex number trong bài báo gốc.

2.3 Nén hạng thời gian

Giả sử các nhãn thực sự xuất hiện là

$$\tau_1 < \tau_2 < \cdots < \tau_r.$$

Thay τ_i bởi i.

Bổ đề 2.3 (Bảo toàn qua nén hạng)**.** *Phép nén hạng tạo song ánh giữa các đường đi thời gian trước và sau biến đổi.*

Proof. Tính khả thi chỉ dùng các so sánh $t_i \le t_{i+1}$. Ánh xạ hạng bảo toàn thứ tự chặt, nên

$$\tau_i \le \tau_j \iff i \le j.$$

Đường đi tĩnh và lựa chọn lần xuất hiện của mỗi cạnh không thay đổi. Ánh xạ thuận và nghịch do đó bảo toàn từng nghiệm. □

Hệ quả thực hành là độ phức tạp nên viết theo r thay vì giá trị số lớn nhất T. Luôn có $r \le |\mathcal{E}|$, kể cả khi timestamp gốc là Unix time hoặc số nguyên rất thưa.

2.4 Rừng, series–parallel và treewidth

Trong rừng, giữa s và z có nhiều nhất một đường đi tĩnh. Điều này loại bỏ phần khó về lựa chọn topology và chỉ còn đếm chuỗi nhãn không giảm.

Đồ thị TTSP được xây dựng đệ quy từ một cạnh có thứ tự hai cực (x, y) bằng hai phép:

- *series*: đồng nhất cực phải của thành phần thứ nhất với cực trái của thành phần thứ hai;
- *parallel*: đồng nhất đồng thời hai cặp cực tương ứng.

Khái niệm series–parallel trong tài liệu có một số biến thể. Khóa luận phân biệt thuật toán TTSP chuyên biệt, yêu cầu truy vấn trùng hai cực, với hệ quả treewidth áp dụng cho partial 2-tree và đầu cuối tùy ý. Việc nhận dạng và phân rã series–parallel có thể thực hiện hiệu quả [4].

Định nghĩa 2.4 (Phân rã cây)**.** Một phân rã cây của $G = (V, E)$ là cây $\mathcal{T}$ với mỗi nút x mang bag $B_x \subseteq V$, thỏa:

1. mỗi đỉnh xuất hiện trong ít nhất một bag;
2. với mọi $uv \in E$, có bag chứa cả u, v;
3. các bag chứa một đỉnh cố định tạo thành cây con liên thông.

Độ rộng là $\max_x |B_x| - 1$; treewidth $\mathrm{tw}(G)$ là độ rộng nhỏ nhất.

Nice tree decomposition dùng các nút leaf, introduce-vertex, introduce-edge, forget-vertex và join. Trong khóa luận, s, z được thêm vào mọi bag; nếu phân rã ban đầu có độ rộng w, kích thước bag sau chuẩn hóa không quá $w + 3$. Mỗi cạnh tĩnh được introduce đúng một lần.

2.5 Kết quả đã biết và khoảng trống nghiên cứu

Bài báo Algorithmica 2025 chứng minh ba dữ kiện định hướng nghiên cứu này [1]:

1. **#Temporal Path** khó trong trường hợp tổng quát và có hardness tham số mạnh;
2. trường hợp đồ thị nền là rừng giải được trong $\mathcal{O}(|V|T^2)$;
3. trường hợp tham số hóa đồng thời bởi treewidth và lifetime là FPT nhờ MSO.

Bài báo kết thúc bằng giả thuyết rằng thuật toán trên rừng mở rộng được sang series–parallel, gợi ý khai thác phép ghép song song. Phiên bản STACS 2023 cung cấp bản tóm lược sớm hơn của hướng nghiên cứu [2].

Khoảng trống có hai tầng. Ở tầng kỹ thuật, chưa có thuật toán thực thi cho series–parallel hoặc DP treewidth tường minh trong công trình gốc. Ở tầng thực nghiệm, bài báo mang tính lý thuyết và không cung cấp bộ benchmark hay mã nguồn tác giả. Vì vậy, khóa luận so sánh trực tiếp với truy hồi forest đã công bố, nhưng không dựng nên một "implementation của tác giả" không tồn tại. Bộ dữ liệu tổng hợp trong khóa luận được gọi là *paper-faithful reproducible benchmark suite*.

2.6 Vị trí của đóng góp mới

Bảng 2.1 phân biệt kết quả gốc và kết quả khóa luận.

Bảng 2.1: Vị trí của các kết quả trong khóa luận.

Thiết lập	Bài báo 2025	Khóa luận
Rừng	$\mathcal{O}(\|V\|T^2)$	$\mathcal{O}(\|P\|r)$; bản thưa tuyến tính theo nhãn liên quan
TTSP, cực là truy vấn	Để mở trong giả thuyết SP	Ma trận truyền $\mathcal{O}(mr^3)$
Treewidth + lifetime	FPT bằng MSO/Courcelle	DP kết nối tường minh
Treewidth riêng lẻ	Không tuyên bố FPT	XP: $Nr^{\mathcal{O}(w)}2^{\mathcal{O}(w \log w)}$
Series–parallel	Giả thuyết đa thức	Hệ quả đa thức từ $w \leq 2$

Chương 3

Thuật toán nhanh trên rừng

3.1 Truy hồi gốc

Nếu s, z không liên thông trong rừng thì đáp án bằng 0. Ngược lại, gọi

$$P = (v_0 = s, v_1, \ldots, v_\ell = z), \qquad e_i = \{v_{i-1}, v_i\}$$

là đường đi tĩnh duy nhất. Mọi đường đi thời gian phải sử dụng đúng dãy cạnh này. Truy hồi của Enright và cộng sự đặt $F(v_0, t) = 1$ và

$$F(v_i, t) = \sum_{\substack{t' \in L(e_i) \\ t' \le t}} F(v_{i-1}, t'). \tag{3.1}$$

Giá trị cần tìm là $F(v_\ell, r)$. Đánh giá trực tiếp mọi cặp (t, t') cho cận xấu nhất $\mathcal{O}(|P| r^2)$. Cài đặt `forest-paper` cố ý giữ nguyên vòng lặp này để làm baseline trung thành với bài báo.

3.2 Quy hoạch động tổng tiền tố

Quan sát rằng tập số hạng trong equation (3.1) tại t chỉ khác tại $t-1$ bởi nhãn đúng bằng t. Do đó

$$F_i(t) = F_i(t-1) + \mathbf{1}[t \in L(e_i)] F_{i-1}(t). \tag{3.2}$$

Thuật toán 1 `forest-dense`: tổng tiền tố dày

```
prev[t] ← 1 với mọi t ∈ [r]
for i = 1, ..., ℓ do
    cur[0] ← 0
    for t = 1, ..., r do
        cur[t] ← cur[t − 1]
        if t ∈ L(e_i) then
            cur[t] ← cur[t] + prev[t]
    swap(prev, cur)
return prev[r]
```

Định lý 3.1 (Cải tiến dày trên rừng)**.** *Sau nén hạng, số đường đi thời gian trên đường đi tĩnh duy nhất P tính được trong $\mathcal{O}(|P|r)$ phép toán số học và $\mathcal{O}(r)$ bộ nhớ làm việc.*

Proof. Phương trình equation (3.2) chính là hiệu giữa hai tổng lồng nhau liên tiếp trong equation (3.1); do đó thuật toán tính cùng giá trị. Mỗi cạnh thực hiện một vòng r bước và chỉ giữ hai hàng DP. □

3.3 Quy hoạch động thưa

Khi r lớn nhưng mỗi cạnh chỉ hoạt động ở vài thời điểm, duyệt toàn bộ miền $[r]$ là lãng phí. Định nghĩa

$$C_i(t) = \#\{\text{tiền tố thời gian đến } v_i \text{ dùng cạnh cuối đúng tại } t\}, \quad t \in L(e_i).$$

Với cạnh đầu tiên, $C_1(t) = 1$. Với $i > 1$,

$$C_i(t) = \sum_{\substack{\tau \in L(e_{i-1}) \\ \tau \le t}} C_{i-1}(\tau). \tag{3.3}$$

Hai danh sách nhãn đã sắp xếp cho phép duyệt bằng một con trỏ và một tổng đang chạy.

Thuật toán 2 Một bước chuyển thưa giữa hai cạnh liên tiếp

```
j ← 0, S ← 0
for all t ∈ L(e_i) theo thứ tự tăng do
    while j < |L(e_{i−1})| và L(e_{i−1})[j] ≤ t do
        S ← S + C_{i−1}[j]; j ← j + 1
    C_i[t] ← S
```

Định lý 3.2 (Cải tiến thưa trên rừng)**.** *Với danh sách nhãn đã sắp xếp, đáp án tính được trong*

$$\mathcal{O}\left(|P| + \sum_{e \in P} |L(e)|\right)$$

phép toán số học, ngoài chi phí sắp xếp hoặc nén hạng.

Proof. Trong mỗi bước, con trỏ qua danh sách trước chỉ tăng và mỗi phần tử của danh sách hiện tại được xử lý một lần. Cộng chi phí trên các cặp cạnh liên tiếp cho cận đã nêu. Tổng các $C_\ell(t)$ là đáp án vì các tập nghiệm phân hoạch theo thời điểm của cạnh cuối. □

3.4 Oracle khép kín

Nếu mọi cạnh của đường đi dài ℓ hoạt động ở mọi hạng $1, \dots, r$, mỗi nghiệm tương ứng một dãy không giảm độ dài ℓ. Theo lập luận stars-and-bars,

$$\#\mathrm{TP}(P_\ell) = \binom{r+\ell-1}{\ell}. \tag{3.4}$$

Đây là oracle quan trọng vì cho phép kiểm tra số nguyên lớn mà không dựa vào một cài đặt DP khác. Các kiểm thử bắt buộc còn gồm một cạnh có hai nhãn, ví dụ thủ công cho đáp án 4, trường hợp thứ tự giảm cho đáp án 0, và hai nhãn bằng nhau phải được chấp nhận.

3.5 Ý nghĩa thuật toán

Cải tiến không thay đổi không gian nghiệm hay giả thiết cấu trúc; nó chỉ đánh giá lại cùng truy hồi theo cách tránh tính lặp tổng tiền tố. Phiên bản dày thích hợp khi hầu hết cạnh hoạt động ở hầu hết thời điểm và có locality bộ nhớ tốt. Phiên bản thưa thích hợp khi mật độ nhãn nhỏ: thời gian phụ thuộc kích thước temporal input thật sự trên đường đi chứ không phụ thuộc tích $|P|r$. Kết quả thực nghiệm ở Chương 7 xác nhận baseline tăng nhanh hơn theo r.

Chương 4

Đại số truyền cho đồ thị TTSP

4.1 Tại sao không thể liệt kê đường đi tĩnh

Một diamond gồm hai nhánh dài hai giữa cùng hai cực. Chuỗi k diamond có 2^k đường đi tĩnh giữa hai cực dù kích thước đồ thị chỉ tuyến tính theo k. Một thuật toán liệt kê từng đường đi rồi đếm nhãn sẽ có thời gian hàm mũ ngay cả khi r nhỏ. Phân rã TTSP cung cấp cách nén các lựa chọn tuyến bằng hai phép hợp thành đại số.

4.2 Ma trận truyền thời gian

Với thành phần TTSP H có hai cực có thứ tự (x, y), định nghĩa ma trận $r \times r$

$$M_H[a,b] = \#\left\{ \begin{array}{l} \text{đường đi thời gian } x\text{–}y \text{ nằm trong } H, \\ \text{nhãn đầu tiên} \geq a, \ \text{nhãn cuối cùng} = b \end{array} \right\}. \tag{4.1}$$

Chỉ số a được hiểu như một điều kiện ngưỡng từ thành phần đứng trước; chỉ số b giữ đủ thông tin để nối sang thành phần tiếp theo.

Với một cạnh $e = xy$,

$$M_e[a,b] = \begin{cases} 1, & b \in L(e) \text{ và } a \leq b, \\ 0, & \text{ngược lại.} \end{cases} \tag{4.2}$$

4.3 Ghép song song

Giả sử $H = H_1 \parallel H_2$. Hai nhánh chỉ chung hai cực. Một đường đi đơn từ cực trái sang cực phải không thể đi vào một nhánh rồi chuyển sang nhánh kia mà không lặp một cực. Do đó, tập nghiệm là hợp rời của tập nghiệm trong hai nhánh:

$$M_H = M_{H_1} + M_{H_2}. \tag{4.3}$$

4.4 Ghép nối tiếp

Giả sử $H = H_1 \circ H_2$, trong đó cực phải của H_1 và cực trái của H_2 được đồng nhất thành q. Mỗi đường đi đơn qua H phân rã duy nhất thành một đường trong H_1 và một đường trong H_2. Nếu phần đầu kết thúc tại hạng k, phần sau phải có nhãn đầu tiên ít nhất k. Chính điều kiện này đã được mã hóa bởi hàng k trong ma trận thứ hai:

$$M_H[a,b] = \sum_{k=1}^{r} M_{H_1}[a,k]\, M_{H_2}[k,b]. \tag{4.4}$$

Vì vậy,

$$M_H = M_{H_1} M_{H_2}.$$

Định lý 4.1 (Thuật toán truyền TTSP)**.** *Cho một phân rã TTSP có hai cực gốc là (s,z). Số đường đi thời gian không nghiêm ngặt từ s đến z tính được chính xác trong $\mathcal{O}(mr^3)$ phép toán số học và $\mathcal{O}(mr^2)$ bộ nhớ với cách lưu trực tiếp mọi ma trận nút.*

Proof. Quy nạp theo cây phân rã. Trường hợp lá đúng theo equation (4.2). Với nút parallel, mọi đường đi nằm trọn trong đúng một nhánh, nên equation (4.3) vừa đầy đủ vừa không đếm trùng. Với nút series, phép tách tại cực chung là duy nhất và tổng trong equation (4.4) liệt kê đúng mọi hạng kết thúc trung gian khả thi. Ở gốc, không có ngưỡng trước đó nên đáp án là

$$\sum_{b=1}^{r} M_G[1,b].$$

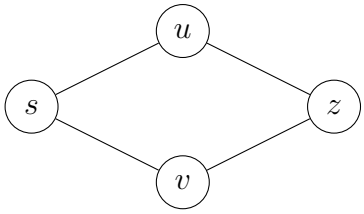
Cây phân rã có $\mathcal{O}(m)$ nút; nhân ma trận thông thường tốn $\mathcal{O}(r^3)$ mỗi nút series. □

4.5 Ví dụ diamond

Chuỗi k diamond có 2^k đường đi tĩnh, mỗi đường dài $2k$. Nếu mọi cạnh hoạt động ở mọi hạng, kết hợp lựa chọn tuyến với equation (3.4) cho oracle

$$\#\mathrm{TP} = 2^k \binom{r+2k-1}{2k}. \tag{4.5}$$

Oracle này kiểm tra được các trường hợp có số tuyến tĩnh quá lớn để brute force nhưng vẫn có đáp án đóng.



$$H = (su \circ uz) \parallel (sv \circ vz)$$

Hình 4.1: Một diamond là phép ghép song song của hai nhánh nối tiếp.

4.6 Giới hạn của thuật toán chuyên biệt

Ma trận truyền sử dụng hướng có thứ tự từ cực trái sang cực phải. Cài đặt yêu cầu cây phân rã nhất quán và hai cực gốc đúng bằng truy vấn (s, z). Nếu truy vấn là hai đỉnh nội bộ tùy ý, phân rã TTSP ban đầu không trực tiếp cung cấp cùng phép hợp thành. Đây là lý do cần thuật toán treewidth ở Chương 5. Hai thuật toán bổ sung nhau: transfer DP có số mũ theo r nhỏ hơn và hiệu quả trên đúng miền cực–cực; treewidth DP tổng quát hơn nhưng trạng thái lớn hơn.

4.7 Hướng tối ưu hóa

Ma trận lá có cấu trúc suffix theo chỉ số a, và nhiều ma trận trung gian có vùng zero. Cài đặt hiện tại bỏ qua tích vô hướng có một thừa số zero nhưng vẫn dùng bố cục dày. Các hướng tiếp theo gồm lưu hàng thưa, nhân block, giải phóng ma trận con sau khi dùng và khảo sát cấu trúc đơn điệu được bảo toàn. Khóa luận không dùng nhân ma trận nhanh để tuyên bố cận thực nghiệm chưa được đo.

Chương 5

Quy hoạch động theo treewidth

5.1 Ý tưởng cục bộ hóa điều kiện thời gian

Khó khăn của đường đi thời gian dường như là điều kiện toàn cục trên cả dãy nhãn. Tuy nhiên, sau khi định hướng đường đi được chọn từ s tới z, điều kiện này là cục bộ tại từng đỉnh trong.

Bổ đề 5.1 (Bổ đề thời gian cục bộ)**.** *Gọi $t_{\text{in}}(v)$ và $t_{\text{out}}(v)$ là nhãn của cạnh vào và cạnh ra tại mỗi đỉnh trong v của một đường đi đã định hướng. Đường đi là đường đi thời gian khi và chỉ khi*

$$t_{\text{in}}(v) \le t_{\text{out}}(v)$$

với mọi đỉnh trong v.

Proof. Nếu dãy nhãn không giảm thì mọi cặp liên tiếp thỏa bất đẳng thức. Ngược lại, nối chuỗi các bất đẳng thức tại các đỉnh liên tiếp cho $t_1 \le t_2 \le \cdots \le t_\ell$. □

Nhờ bổ đề, DP không cần giữ toàn bộ dãy nhãn của một thành phần. Nó chỉ giữ nhãn của một cạnh duy nhất tại đầu mút đang thiếu cạnh còn lại. Khi cả cạnh vào và ra đã biết, so sánh được thực hiện và hai nhãn có thể quên.

5.2 Chuẩn hóa phân rã

Cho phân rã cây độ rộng w. Thêm s, z vào mọi bag, tăng độ rộng nhiều nhất 2, rồi chuyển về nice tree decomposition gồm leaf, introduce-vertex, introduce-edge, forget-vertex và join. Mỗi cạnh tĩnh được introduce đúng một lần. Khi đó kích thước bag

$$b \le w + 3.$$

Việc giữ hai terminal trong mọi bag đơn giản hóa điều kiện gốc và bảo đảm thành phần nghiệm cuối cùng không biến mất khỏi separator.

5.3 Bất biến nghiệm bộ phận

Tại nút x, gọi E_x là các cạnh đã introduce trong cây con của x. Một realization bộ phận chọn một tập con E_x, một hướng và một hạng hoạt động cho mỗi cạnh đã chọn. DP duy trì năm bất biến:

I1. Bậc vào và bậc ra của mỗi đỉnh không quá một.

I2. Đồ thị tĩnh được chọn là rừng.

I3. Khi cả hai cạnh đường đi tại một đỉnh đã biết, bất đẳng thức thời gian cục bộ đã được kiểm tra.

I4. Mọi đỉnh được dùng nhưng đã forget có đúng một cạnh vào và một cạnh ra.

I5. Mọi thành phần được chọn vẫn giao với bag hiện tại; không thành phần nào biến mất sau separator.

5.4 Không gian trạng thái

Với mỗi đỉnh bag v, lưu một trong các trạng thái

$$\text{Zero}, \quad \text{In}(t), \quad \text{Out}(t), \quad \text{Both}.$$

$\text{In}(t)$ nghĩa là đã có đúng một cạnh vào tại t; Out tương tự. Both nghĩa là đã có một cạnh vào và một cạnh ra, đồng thời kiểm tra thời gian đã thành công. Không cần nhớ nhãn sau khi đạt Both, vì không cạnh mới nào còn được phép kề với đỉnh.

Ngoài chữ ký cục bộ σ, trạng thái lưu phân hoạch Π của các đỉnh active (không phải Zero). Hai đỉnh thuộc cùng block khi chúng nằm trong cùng một thành phần của rừng bộ phận. Trạng thái đầy đủ là (σ, Π) và giá trị bảng là số realization chính xác.

5.5 Các chuyển trạng thái đơn

5.5.1 Leaf và introduce-vertex

Ở leaf, mọi đỉnh bag có trạng thái Zero, phân hoạch rỗng và count 1. Đỉnh vừa introduce cũng bắt đầu ở Zero.

5.5.2 Introduce-edge

Với cạnh uv, DP có lựa chọn bỏ qua, hoặc chọn $u \to v$ hay $v \to u$ tại mỗi $t \in L(uv)$. Thêm cạnh ra tại t cập nhật

$$\begin{aligned} \text{Zero} &\to \text{Out}(t), \\ \text{In}(a) &\to \text{Both} \quad \text{nếu } a \le t, \end{aligned}$$

và các trường hợp khác không hợp lệ. Thêm cạnh vào cập nhật đối xứng:

$$\begin{aligned} \text{Zero} &\to \text{In}(t), \\ \text{Out}(a) &\to \text{Both} \quad \text{nếu } t \le a. \end{aligned}$$

Nếu hai đầu đã active và cùng block, chọn cạnh sẽ tạo chu trình nên bị loại; nếu không, các block được hợp nhất hoặc tạo mới.

5.5.3 Forget-vertex

Chỉ nonterminal được forget. Zero và Both được chấp nhận; In hoặc Out bị loại vì đỉnh sẽ không còn cơ hội nhận cạnh thiếu. Nếu đỉnh là đại diện bag cuối cùng của block thì trạng thái cũng bị loại, bảo toàn bất biến I5.

5.6 Nút join và bổ đề incidence

Hai con của join có cùng bag và hai tập cạnh đã introduce rời nhau. Chữ ký cục bộ được ghép theo bảng:

Cặp	Kết quả
$\text{Zero} + X$	X
$\text{In}(a) + \text{Out}(b)$	Both nếu $a \le b$
$\text{Out}(b) + \text{In}(a)$	Both nếu $a \le b$
$\text{In} + \text{In},\ \text{Out} + \text{Out}$	không hợp lệ
$\text{Both} + \text{nonzero}$	không hợp lệ.

Phần tinh tế là phát hiện chu trình khi hợp hai rừng con. Xây dựng incidence multigraph hai phía J: mỗi thành phần của rừng con trái chạm bag là một nút trái, mỗi thành phần con phải là một nút phải; mỗi đỉnh bag active ở cả hai con tạo một cạnh incidence nối hai thành phần chứa nó.

Bổ đề 5.2 (Kiểm tra chu trình tại join)**.** *Hợp $F_1 \cup F_2$ là rừng khi và chỉ khi*

incidence multigraph J không có chu trình. Hai cạnh incidence song song được tính là chu trình độ dài hai.

Proof. Nếu J có chu trình, hai đỉnh bag liên tiếp trên chu trình nằm trong cùng một thành phần cây của một con. Nối các đường đi duy nhất bên trong các cây con theo chu trình incidence tạo một chu trình trong hợp. Hai cạnh song song cho hai đường nội bộ khác nhau giữa cùng hai đỉnh bag.

Ngược lại, một chu trình của $F_1 \cup F_2$ không thể nằm hoàn toàn trong một con vì mỗi con là rừng. Chia nó thành các đoạn tối đại luân phiên giữa hai con và co mỗi đoạn về thành phần con chứa nó. Ta thu được closed walk trong J, nên J chứa chu trình. □

Cài đặt dùng union–find nhỏ trên các block trái/phải. Khi thêm một cạnh incidence mà hai đầu đã liên thông, trạng thái bị loại ngay. Nếu hợp lệ, phân hoạch cha là bao đóng bắc cầu của hai quan hệ liên thông con, và count cha tăng bởi tích hai count.

5.7 Điều kiện chấp nhận

Ở gốc, mọi nonterminal đã được forget. Chấp nhận đúng các trạng thái

$$s = \mathrm{Out}(a), \qquad z = \mathrm{In}(b), \qquad \Pi = \{\{s, z\}\}.$$

Tổng trên mọi a, b là đáp án.

5.8 Chứng minh đúng

Định lý 5.3 (Tính đúng của DP treewidth)**.** *DP đếm mỗi đường đi thời gian không nghiêm ngặt từ s tới z đúng một lần và không đếm đối tượng nào khác.*

Soundness. Mọi realization được chấp nhận là acyclic; các đỉnh trong được dùng có bậc vào và ra bằng một; s, z có bậc một; và chỉ tồn tại một thành phần chứa s, z. Bất biến không biến mất loại mọi thành phần ẩn. Vì vậy đồ thị được chọn chính xác là một đường đi tĩnh đơn. Điều kiện bậc/hướng buộc hướng từ s tới z. Mọi đỉnh trong đã vượt qua so sánh nhãn cục bộ; theo bổ đề thời gian cục bộ, toàn bộ dãy nhãn không giảm. □

Completeness và không đếm trùng. Lấy một đường đi thời gian bất kỳ. Tại nút introduce duy nhất của mỗi cạnh trên đường, chọn đúng cạnh, hướng từ s tới z và nhãn của nghiệm; bỏ qua mọi cạnh khác. Tính đơn bảo đảm kiểm tra bậc và chu trình không thất bại; tính thời gian bảo đảm mọi so sánh thành công. Tính liên thông của các bag chứa một đỉnh trong phân rã cây bảo đảm cả hai cạnh của đỉnh đã được xử lý trước khi nó bị forget. Do mỗi cạnh được introduce đúng một lần, đường đi xác định duy nhất toàn bộ dãy lựa chọn, nên không đếm trùng. □

5.9 Độ phức tạp

Mỗi đỉnh bag có $2r+2$ trạng thái cục bộ. Số phân hoạch của tối đa b phần tử là số Bell B_b, với $B_b \le b^b = 2^{\mathcal{O}(b \log b)}$. Do đó

$$S_b \le (2r+2)^b B_b.$$

Cài đặt join đơn giản thử mọi cặp trạng thái, cho cận an toàn

$$\mathcal{O}\big(N(2r+2)^{2(w+3)} B_{w+3}^2 \,\mathrm{poly}(w)\big) = N r^{\mathcal{O}(w)} 2^{\mathcal{O}(w \log w)}.$$

Vì $r \le |\mathcal{E}| \le N$, đây là $N^{\mathcal{O}(w)}$.

Hệ quả 5.4 (XP theo treewidth)**.** ***#Temporal Path*** *thuộc XP khi tham số là treewidth của đồ thị nền và một phân rã độ rộng w được cung cấp.*

Hệ quả 5.5 (Series–parallel)**.** ***#Temporal Path*** *giải được trong thời gian đa thức trên đồ thị series–parallel/partial 2-tree, kể cả khi s, z là hai đỉnh tùy ý.*

Proof. Các đồ thị này có treewidth không quá 2. Thay $w = 2$ vào cận XP cho một đa thức theo kích thước đầu vào. □

Hệ quả cuối là câu trả lời đề xuất cho giả thuyết series–parallel của Enright và cộng sự. Cận thô chứa lũy thừa cao theo r và không được quảng bá như một thuật toán SP tối ưu; khi truy vấn trùng hai cực TTSP, thuật toán ở Chương 4 thực tế hơn.

Chương 6

Thiết kế và cài đặt hệ thống

6.1 Mục tiêu kỹ thuật

Cài đặt phục vụ đồng thời ba mục tiêu: kiểm tra chứng minh, đo hiệu năng và tái lập kết quả. Vì sai lệch một dấu so sánh hoặc một phép ghép phân hoạch có thể tạo kết quả có vẻ hợp lý trên nhiều dữ liệu, ưu tiên đầu tiên là tính độc lập của oracle và khả năng dừng ngay tại phản ví dụ.

Mã nguồn dùng C++20 cho solver và Python chỉ cho sinh dữ liệu, điều phối benchmark và tổng hợp kết quả. Các solver dùng chung parser và kiểu số nguyên lớn nhưng không chia sẻ logic chuyển trạng thái với phương pháp tham chiếu.

6.2 Kiến trúc repository

Bảng 6.1: Các thành phần chính của phần mềm.

Thành phần	Trách nhiệm
src/temporal_graph.*	Parse, loại trùng, xây đồ thị nền và nén hạng
src/bigint.hpp	Số nguyên không âm độ dài tùy ý, limb 32 bit
src/solvers.*	Ba solver forest và static-path reference
src/ttsp.*	Parse cây SP và đánh giá ma trận truyền
src/tw_dp.*	Nice-TD validator và DP kết nối
src/main.cpp	CLI, timing, RSS và đầu ra JSON
python/generators.py	Sinh forest, TTSP, partial k-tree và decomposition
python/audit.py	Chiến dịch so khớp xác định, dừng khi mismatch
python/benchmark.py	Grid hiệu năng và schema CSV
tests/	Unit test thủ công và fixture decomposition

6.3 Biểu diễn đồ thị và nén hạng

Định dạng canonical có header

```
p n m_temporal s z
u v t
```

```
...
```

Cạnh được xem là vô hướng bằng cách chuẩn hóa (u, v) thành $(\min(u, v), \max(u, v))$. Parser từ chối self-loop và chỉ số đỉnh ngoài miền, kiểm tra số dòng theo header, loại các triple trùng và nén toàn bộ timestamp phân biệt về $1, \ldots, r$. Danh sách nhãn của mỗi cạnh tăng dần theo construction.

6.4 Số nguyên chính xác

Đáp án có thể vượt 64 bit ngay trên diamond-chain vừa phải. Repository cung cấp kiểu số nguyên không âm tự chứa, lưu little-endian theo cơ số 2^{32}. Nó hỗ trợ cộng, nhân dài, nhân/chia số nhỏ chính xác, xuất thập phân và tính bit length. Việc không phụ thuộc Boost làm bản build tái lập trên môi trường tối giản; đồng thời unit test và sanitizer được dùng để giảm rủi ro của thành phần số học tự cài đặt.

6.5 Mã hóa trạng thái treewidth

Trạng thái cục bộ được mã hóa bằng số nguyên:

$$0 = \text{Zero}, \quad 1..r = \text{In}(t), \quad r+1..2r = \text{Out}(t), \quad 2r+1 = \text{Both}.$$

Phân hoạch được canonical hóa dưới dạng restricted-growth-like labels: duyệt theo thứ tự bag và đổi block đầu tiên thành 0, block mới tiếp theo thành 1, v.v. Nhờ đó, hai union–find tạo cùng phân hoạch có cùng hash key. Bảng chỉ lưu trạng thái reachable trong `unordered_map`; không cấp phát toàn bộ không gian lý thuyết.

Nice-TD parser kiểm tra các điều kiện dễ gây lỗi: cấu trúc là cây có gốc, s, z nằm trong mọi bag, root đúng bằng $\{s, z\}$, bag trước/sau introduce/forget hợp lệ, và mỗi cạnh tĩnh được introduce đúng một lần.

6.6 CLI và telemetry

Một executable duy nhất cung cấp các lệnh như:

```
tpcount count --solver forest-paper instance.tg
tpcount count --solver forest-dense instance.tg
tpcount count --solver forest-sparse instance.tg
```

```
tpcount count --solver ttsp-transfer instance.tg --decomp instance.
   sp
tpcount count --solver tw-dp instance.tg --td instance.ntd
tpcount verify --solver tw-dp instance.tg --td instance.ntd
```

Đầu ra JSON chứa solver, đáp án thập phân, bit length, wall time, peak RSS, kích thước input, số trạng thái tạo ra, peak state per bag và số cặp join đã thử. Đáp án được lưu thêm SHA-256 trong CSV benchmark để so khớp giá trị lớn mà không cần tải toàn bộ chuỗi vào công cụ tổng hợp.

6.7 Phương pháp tham chiếu độc lập

`brute` dùng DFS liệt kê mọi đường đi tĩnh đơn từ s tới z. Trên mỗi đường, một label-DP nhỏ đếm chuỗi không giảm. Cấu trúc này độc lập với decomposition DP và transfer algebra, nên phù hợp để phát hiện lỗi ở hai solver chính. Các oracle đóng equations (3.4) and (4.5) tạo lớp kiểm tra thứ ba không dựa vào enumeration.

6.8 Sinh dữ liệu và seed

Master seed là 20260928. Seed của mỗi instance là số nguyên 64 bit từ tám byte đầu của

$$\mathrm{SHA256}(\texttt{"20260928:family:grid_cell:replicate"}).$$

Cách lập lịch này tránh thay đổi ngầm do thêm bớt một cell trước đó. Bộ sinh hỗ trợ: tất cả nhãn; đúng q nhãn chọn đều không hoàn lại; và Bernoulli mật độ p. Với Bernoulli, nếu một cạnh không nhận nhãn, hệ thống ép một hạng ngẫu nhiên đều để bảo toàn topology tĩnh.

Partial k-tree được sinh cùng phân rã đã biết. Nhờ vậy, thời gian tìm decomposition không lẫn vào solver timing và giá trị width không bị báo nhầm từ heuristic.

6.9 Quy trình thất bại

Nếu có mismatch, benchmark dừng ngay, lưu graph, nhãn, (s, z), decomposition, seed và output của các solver. Bước tiếp theo là delta-debug bằng cách loại đỉnh/cạnh/nhãn và giảm r cho đến phản ví dụ tối tiểu. Không

chạy hiệu năng tiếp khi còn sai lệch chưa giải thích. Đây là release gate quan trọng hơn mọi tối ưu.

Chương 7

Đánh giá thực nghiệm

7.1 Câu hỏi và giả thuyết thực nghiệm

Thực nghiệm đánh giá bốn giả thuyết:

- **H1**: baseline forest tăng gần bậc hai theo r, trong khi dense-prefix tăng gần tuyến tính;
- **H2**: ở nhãn thưa, thời gian của sparse solver được giải thích tốt hơn bởi $\sum |L(e)|$ so với $|P|r$;
- **H3**: transfer DP nén được số tuyến tăng theo hàm mũ trên diamond-chain;
- **H4**: treewidth DP cho đầu cuối tùy ý đồng ý với brute force trên partial 2-tree nhỏ.

7.2 Môi trường

Các số liệu hiện tại được đo trên Intel Core i9-9980HK, 8 nhân vật lý/16 luồng, RAM 15 GiB, Linux 6.12.93 x86-64, GCC 14.3.0 và Python 3.13.12. C++ được biên dịch với `-std=c++20 -O3 -DNDEBUG`. Mỗi solver chạy một tiến trình; sinh graph và decomposition nằm ngoài timed region. Peak RSS lấy từ `getrusage`.

Các số liệu forest smoke là một lượt đo kỹ thuật, chưa phải số cuối để xuất bản. Grid TTSP bắt buộc đã chạy toàn bộ. Correctness audit dùng dữ liệu xác định và phương pháp tham chiếu độc lập.

7.3 Kiểm chứng tính đúng

Bảng 7.1 tổng hợp chiến dịch audit mặc định.

Ngoài ra, một chiến dịch riêng lưu $1,000$ đối chiếu partial 2-tree vào CSV chi tiết. Unit test bao phủ single edge, thứ tự thời gian bị từ chối,

Bảng 7.1: Kết quả đối chiếu chính xác với phương pháp tham chiếu.

Họ instance	Số đối chiếu	Mismatch
Rừng ngẫu nhiên	3,000	0
TTSP đệ quy ngẫu nhiên	2,500	0
Partial 2-tree, truy vấn tùy ý	1,000	0
Tổng	**6,500**	**0**

timestamp bằng nhau, công thức đường đi, diamond, triangle và một decomposition có join thật. Bản sanitizer AddressSanitizer/UndefinedBehaviorSanitizer hoàn thành unit test mà không phát hiện lỗi runtime; leak detection bị tắt do giới hạn ptrace của môi trường.

Zero mismatch không chứng minh thuật toán đúng, nhưng tăng độ tin cậy của sự tương ứng giữa chứng minh và mã. Đặc biệt, tập partial 2-tree có nhiều join và cặp đầu cuối không nhất thiết là cực TTSP.

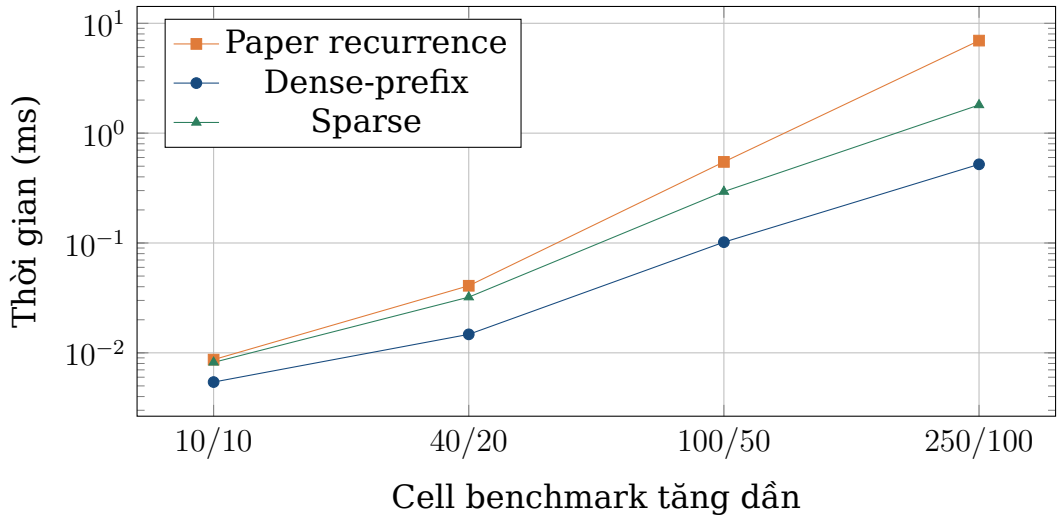
7.4 So sánh forest

Bảng 7.2 báo cáo bốn cell dày. Vì mỗi cell chỉ chạy một lần và thời gian nhỏ, chúng chỉ minh họa xu hướng.

Bảng 7.2: Forest dense smoke benchmark.

$\lvert P\rvert$	r	Paper (ms)	Dense (ms)	Sparse (ms)	P/D	P/S
10	10	0.00868	0.00542	0.00819	1.60	1.06
40	20	0.0408	0.0147	0.0321	2.77	1.27
100	50	0.547	0.102	0.294	5.38	1.86
250	100	6.96	0.520	1.81	13.37	3.85

Speedup tăng theo kích thước cell, phù hợp H1. Trên dữ liệu dày, dense-prefix nhanh hơn sparse vì mảng Boolean và truy cập liên tục có hằng số nhỏ hơn; đây không phủ định H2, vốn cần grid giữ r lớn và thay đổi số nhãn mỗi cạnh. Chiến dịch sparse đầy đủ vẫn cần chạy lặp trước khi đưa ra kết luận định lượng.



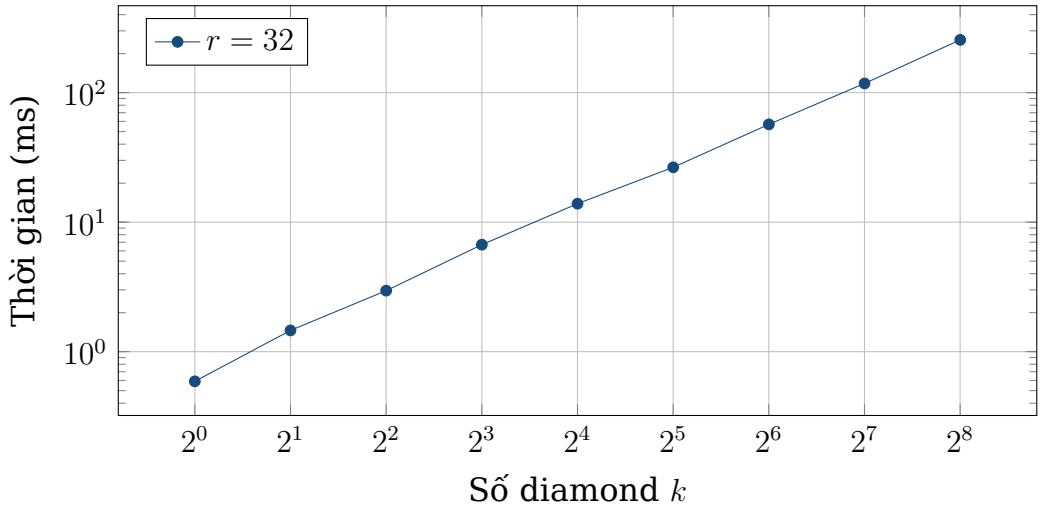
Hình 7.1: Xu hướng thời gian forest; nhãn trục x là $|P|/r$. Trục y logarithmic.

7.5 Diamond-chain TTSP

Grid bắt buộc gồm

$$k \in \{1, 2, 4, 8, 16, 32, 64, 128, 256\}, \quad r \in \{2, 4, 8, 16, 32\}$$

ở chế độ mọi cạnh hoạt động tại mọi hạng. Cả 45 cell transfer đều hoàn thành; các cell nhỏ đồng ý với brute force. Trường hợp lớn nhất $k = 256, r = 32$ có $1,024$ cạnh tĩnh và $32,768$ cạnh–thời gian, trả đáp án 424 bit trong 255.5 ms với peak RSS 87.4 MiB.



Hình 7.2: Thời gian TTSP theo k tại $r = 32$, lấy trực tiếp từ bảng CSV.

Số đường đi tĩnh tại $k = 256$ là 2^{256}, nên enumeration không khả thi. Transfer DP vẫn xử lý theo kích thước decomposition, hỗ trợ H3. Phiên

bản nộp cuối nên tiếp tục sinh hình tự động từ CSV để loại nguy cơ chép số thủ công.

7.6 Treewidth DP

Trong tập $1,012$ hàng partial-2-tree được tổng hợp từ smoke và chiến dịch chi tiết, median wall time của `tw-dp` là khoảng 0.314 ms và lớn nhất khoảng 13.0 ms; brute force có median 0.0087 ms trên kích thước nhỏ. Brute force nhanh hơn ở đây vì instance được cố ý giữ rất nhỏ để làm oracle. Mục tiêu không phải chứng minh ưu thế tốc độ ở miền này mà là xác nhận H4. Khi số đường đi tĩnh tăng, brute force sẽ mất khả năng mở rộng.

Telemetry của fixture diamond có 107 trạng thái tạo ra, peak 25 trạng thái/bag và 169 cặp join. Việc lưu các chỉ số này cho phép các nghiên cứu tiếp theo đánh giá join bucketing thay vì chỉ đo wall time.

7.7 Đe dọa tới tính hợp lệ

Tính hợp lệ nội tại. Một kiểu số nguyên lớn tự cài đặt có thể chứa lỗi; rủi ro được giảm bằng oracle đóng, brute force và sanitizer nhưng chưa có formal verification. Timing cực nhỏ nhạy với scheduling, DVFS và chi phí khởi động tiến trình.

Tính hợp lệ cấu trúc. Dữ liệu tổng hợp phản ánh đúng cấu trúc theorem nhưng không đại diện cho mọi mạng thời gian thực tế. Đồ thị thực thường có treewidth cao; không thể suy rộng kết quả SP sang toàn bộ mạng.

Tính hợp lệ kết luận. $6,500$ đối chiếu không thay thế chiến dịch Graph Atlas và exhaustive temporal labeling. Đặc biệt, các cấu hình join hiếm có thể chưa được bao phủ đủ. Không có benchmark tác giả để so sánh runtime với bài báo 2025; chỉ baseline truy hồi là so sánh trực tiếp công bằng.

Tính mới. Tìm kiếm hiện tại chưa phát hiện công trình sau năm 2025 giải cùng giả thuyết, nhưng một novelty review hệ thống qua DBLP, Google Scholar Cited By, Semantic Scholar, Scopus/Web of Science, arXiv và trang tác giả vẫn cần thực hiện trước khi nộp tuyên bố cuối.

7.8 Kết luận thực nghiệm

Kết quả hiện tại ủng hộ tính đúng của ba họ thuật toán và xu hướng phân tích: prefix DP cải thiện rõ khi r tăng; transfer DP nén số tuyến hàm mũ; treewidth DP xử lý đầu cuối tùy ý trên partial 2-tree và đồng ý với oracle. Tuy nhiên, dữ liệu forest cần lặp và pin CPU, H2 cần grid sparse đầy đủ, và release gate exhaustive vẫn chưa hoàn tất.

Chương 8

Kết luận và hướng phát triển

8.1 Tổng kết kết quả

Khóa luận bắt đầu từ câu hỏi liệu cấu trúc của đồ thị nền có thể làm cho việc đếm chính xác đường đi thời gian trở nên khả thi hay không. Câu trả lời thu được gồm ba tầng liên tiếp.

Trên rừng, phần lựa chọn topology biến mất vì chỉ có một đường đi tĩnh. Truy hồi đã biết chứa cấu trúc tổng tiền tố, cho phép giảm $\mathcal{O}(|P|r^2)$ xuống $\mathcal{O}(|P|r)$. Khi nhãn thưa, hai con trỏ trên danh sách đã sắp xếp cho thời gian tuyến tính theo số nhãn thực sự liên quan.

Trên TTSP với truy vấn cực–cực, ma trận truyền nén đồng thời số đường đi tĩnh và điều kiện thời gian. Hai phép dựng graph tương ứng chính xác với cộng và nhân ma trận. Kết quả là thuật toán $\mathcal{O}(mr^3)$ không phụ thuộc vào số tuyến tĩnh, dù số tuyến có thể là $2^{\Theta(m)}$.

Trên treewidth bị chặn, bổ đề thời gian cục bộ cho phép kết hợp điều kiện nhãn với DP liên thông chuẩn. Trạng thái chỉ cần nhớ cạnh đơn độc tại boundary và quên nhãn khi đỉnh hoàn tất. Phân hoạch connectivity cùng incidence-cycle test tại join bảo đảm nghiệm cuối là một đường đi đơn thay vì hợp các đường hoặc chu trình. Thuật toán chạy trong $Nr^{\mathcal{O}(w)}2^{\mathcal{O}(w \log w)}$, đặt bài toán vào XP theo treewidth.

8.2 Trả lời câu hỏi nghiên cứu

RQ1. Có. Truy hồi forest đánh giá được trong $\mathcal{O}(|P|r)$, và bản thưa đạt $\mathcal{O}(|P| + \sum |L(e)|)$ sau sắp xếp.

RQ2. Có. Transfer matrix với ngữ nghĩa ngưỡng đầu/nhãn cuối biến parallel thành cộng và series thành nhân, cho thuật toán chính xác $\mathcal{O}(mr^3)$.

RQ3. Có, dưới dạng XP chứ không phải FPT. DP tường minh chỉ tham số hóa bởi underlying treewidth. Hệ quả $w \leq 2$ đưa ra thuật toán đa thức cho series–parallel với đầu cuối tùy ý.

RQ4. Bằng chứng hiện có là tích cực: $6\,500$ đối chiếu ngẫu nhiên không mismatch; oracle đường đi và diamond đều đúng; grid diamond lớn nhất hoàn thành; forest speedup tăng theo kích thước. Tuy nhiên, benchmark forest cần lặp và exhaustive audit vẫn chưa hoàn tất.

8.3 Đóng góp đối với giả thuyết series–parallel

Hai con đường giải quyết bổ sung nhau. Transfer DP hiện thực trực tiếp ý tưởng parallel composition mà bài báo gốc gợi ý, nhưng chỉ cho cực–cực. Treewidth DP bao phủ truy vấn tùy ý và chứng minh tính đa thức qua $w = 2$. Vì vậy, ở mức chứng minh và cài đặt hiện tại, khóa luận đã đưa ra lời giải cho trường hợp mở series–parallel.

Tuyên bố học thuật cuối cùng vẫn cần hai kiểm tra bên ngoài: (i) audit chứng minh, đặc biệt tại join và thuật ngữ series–parallel; (ii) novelty review trên các cơ sở dữ liệu cập nhật. Cho đến khi hoàn tất, kết quả nên được mô tả là “giải giả thuyết, với điều kiện thẩm định độc lập”.

8.4 Hạn chế

- Cận XP có số mũ cao và join hiện thử cặp trạng thái; solver phù hợp chủ yếu với width nhỏ và r vừa phải.
- Cài đặt giữ bảng của mọi nút decomposition, chưa giải phóng theo postorder, nên memory có thể cải thiện.
- Chưa có decomposition solver cho graph bên ngoài; input TD được cung cấp hoặc sinh cùng graph.
- Chưa bao phủ strict temporal paths, temporal walks, restless paths hoặc delay-robust routes.
- Chưa chạy Graph Atlas và exhaustive temporal labeling đến $n \leq 5$; chưa có đánh giá trên dữ liệu thực treewidth thấp.
- Tập forest timing hiện tại là smoke run; chưa có median/IQR đủ cho hình cuối.

8.5 Hướng phát triển

Ưu tiên gần nhất là hoàn tất correctness release gate và gửi proof cho phản biện độc lập. Về kỹ thuật, join bucketing theo chữ ký bậc, precompute compatibility, compact partition encoding và giải phóng bảng con có thể giảm đáng kể thời gian/bộ nhớ. Đối với TTSP, cần khai thác vùng zero và cấu trúc suffix của ma trận thay vì nhân dày tổng quát.

Một hướng lý thuyết quan trọng là tìm cận tốt hơn riêng cho series–parallel với truy vấn tùy ý, tránh đa thức bậc cao suy ra từ DP treewidth. Hướng khác là kiểm tra biến thể strict bằng cách thay $\leq$ thành $<$ ở mọi transition và xây lại audit. Cuối cùng, việc trích các component/treewidth-bounded subgraph từ dữ liệu SocioPatterns hoặc SNAP có thể đánh giá external validity, với điều kiện báo cáo rõ tỷ lệ đỉnh/cạnh được bao phủ và không tuyên bố giải graph có treewidth cao.

8.6 Kết lời

Kết quả cho thấy ranh giới giữa “khó trong tổng quát” và “khả thi trên cấu trúc” có thể khai thác bằng những trạng thái rất cụ thể. Nén hạng loại bỏ độ lớn timestamp, prefix scan loại bỏ phép tính lặp, transfer algebra loại bỏ liệt kê tuyến, và separator nhỏ loại bỏ nhu cầu nhớ toàn bộ đường đi. Sự kết hợp giữa chứng minh, oracle độc lập và phần mềm tái lập là nền tảng để chuyển một câu hỏi mở thành một kết quả có thể kiểm tra.

Chương A

Định dạng dữ liệu và decomposition

A.1 Temporal graph

Ví dụ:

```
p 3 4 0 2
0 1 1
0 1 2
1 2 2
1 2 3
```

Header lần lượt là số đỉnh, số dòng temporal edge, nguồn và đích. Đỉnh đánh số từ 0. Timestamp có thể là số nguyên bất kỳ và được nén hạng khi đọc.

A.2 TTSP expression

File SP dùng node ID và root:

```
sp 7 6
0 edge 0 1
1 edge 1 3
2 series 0 1
3 edge 0 2
4 edge 2 3
5 series 3 4
6 parallel 2 5
```

Node phải tạo thành DAG biểu thức thực tế là cây; cực có thứ tự của hai con series phải gặp nhau và hai con parallel phải có cùng cực.

A.3 Nice tree decomposition

Header có số node, root và width của decomposition đầu vào trước chuẩn hóa. Các dạng dòng:

```
ntd NODE_COUNT ROOT WIDTH
ID leaf BAG_SIZE BAG...
ID introduce-v CHILD V BAG_SIZE BAG...
ID introduce-e CHILD U V BAG_SIZE BAG...
ID forget-v CHILD V BAG_SIZE BAG...
ID join LEFT RIGHT BAG_SIZE BAG...
```

Cài đặt yêu cầu s, z trong mọi bag và root đúng bằng $\{s, z\}$. Mỗi static edge phải xuất hiện trong đúng một node `introduce-e`.

A.4 Schema kết quả

CSV chứa family, instance ID, seed, solver, status, n, số cạnh tĩnh/thời gian, r, treewidth, terminal, đáp án, bit length, wall/CPU time, RSS, số trạng thái, join pairs, answer SHA-256, commit, compiler, flags và machine ID. Nếu repository chưa được khởi tạo Git, trường commit có giá trị `unknown`; cần tạo commit trước chiến dịch chính thức.

Chương B

Hướng dẫn tái lập

B.1 Build

Yêu cầu GCC/Clang hỗ trợ C++20 và Python 3.10 trở lên.

```
bash scripts/build.sh
```

Trên máy có CMake backend:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DTPCOUNT_NATIVE=ON
cmake --build build -j
ctest --test-dir build --output-on-failure
```

B.2 Chiến dịch

```
# Smoke benchmark
python3 python/benchmark.py --suite smoke

# 3,000 forest + 2,500 TTSP + 1,000 partial-2-tree
python3 python/audit.py

# Grid diamond ắbt ộbuc
python3 python/benchmark.py --suite ttsp --thesis

# Grid forest đầy đủ(có ểth timeout ởbaseline)
python3 python/benchmark.py --suite forest --thesis

# ổTng ợhp
python3 python/aggregate.py results/*.csv --output results/
   aggregate.md
```

B.3 Biên dịch khóa luận

Repository cung cấp script Tectonic:

```
bash thesis/build_pdf.sh
```

Trước khi nộp, thay các macro thông tin cá nhân ở đầu `thesis/main.tex`, cập nhật `CITATION.cff`, tạo Git commit sạch, và chạy lại toàn bộ bảng/hình trên máy được mô tả trong khóa luận.

Chương C

Checklist thẩm định chứng minh

- ☐ Định nghĩa temporal path khớp bài báo 2025; điều kiện là $\leq$, không phải $<$.
- ☐ Mỗi static edge được introduce đúng một lần và chỉ chọn tối đa một timestamp.
- ☐ Ràng buộc indegree/outdegree ngăn branching.
- ☐ Cycle test tại introduce-edge đúng.
- ☐ Đỉnh nonterminal được dùng phải là Both trước khi forget.
- ☐ Không thành phần nào biến mất khỏi separator.
- ☐ Incidence multigraph lemma đúng, bao gồm parallel incidence edges.
- ☐ Phân hoạch cha ở join là bao đóng đúng của hai quan hệ con.
- ☐ Root loại mọi thành phần dư và buộc đúng một đường s–z.
- ☐ Hướng của đường bắt buộc từ s tới z.
- ☐ Bất đẳng thức cục bộ suy ra dãy nhãn không giảm toàn cục.
- ☐ Completeness sử dụng đúng tính liên thông của bag chứa một đỉnh.
- ☐ Không có double count do edge introduction duy nhất.
- ☐ State bound tính cả Bell partitions.
- ☐ Độ dài bit của số đếm là đa thức với w cố định.
- ☐ Thuật ngữ TTSP, series–parallel và partial 2-tree được phân biệt đúng.
- ☐ Novelty claim đã được kiểm tra độc lập và cập nhật đến ngày nộp.

Không đánh dấu một mục chỉ dựa trên unit test. Checklist này dành cho giảng viên hướng dẫn hoặc phản biện độc lập đọc chứng minh và đối chiếu mã nguồn.

Tài liệu tham khảo

[1] J. Enright, K. Meeks, and H. Molter, “Counting Temporal Paths,” *Algorithmica*, vol. 87, pp. 736–782, 2025. doi: 10.1007/s00453-025-01301-3.

[2] J. Enright, K. Meeks, and H. Molter, “Counting Temporal Paths,” in *40th International Symposium on Theoretical Aspects of Computer Science (STACS 2023)*, LIPIcs, vol. 254, Article 30, 2023. doi: 10.4230/LIPIcs.STACS.2023.30.

[3] B. Courcelle and J. Engelfriet, *Graph Structure and Monadic Second-Order Logic: A Language-Theoretic Approach*. Cambridge University Press, 2012.

[4] D. Eppstein, “Parallel recognition of series-parallel graphs,” *Information and Computation*, vol. 98, no. 1, pp. 41–55, 1992.

[5] O. Michail, “An introduction to temporal graphs: An algorithmic perspective,” *Internet Mathematics*, vol. 12, no. 4, pp. 239–280, 2016.

[6] P. Holme and J. Saramäki, *Temporal Network Theory*. Springer, 2019.

[7] D. Kempe, J. Kleinberg, and A. Kumar, “Connectivity and inference problems for temporal networks,” *Journal of Computer and System Sciences*, vol. 64, no. 4, pp. 820–842, 2002.

[8] S. Buß, H. Molter, R. Niedermeier, and M. Rymar, “Algorithmic aspects of temporal betweenness,” *Network Science*, vol. 12, no. 2, pp. 160–188, 2024.

[9] A. Casteigts, A.-S. Himmel, H. Molter, and P. Zschoche, “Finding temporal paths under waiting time constraints,” *Algorithmica*, vol. 83, no. 9, pp. 2754–2802, 2021.

[10] L. G. Valiant, “The complexity of enumeration and reliability problems,” *SIAM Journal on Computing*, vol. 8, no. 3, pp. 410–421, 1979.